

Section 03 - Rendering Conditional Content and Lists

Rendering Content Dynamically Using `v-if` and `v-else`

`v-if` and `v-else` adds and removes element completely from DOM dynamically

The screenshot displays a development environment with a code editor, a browser preview, and a console window.

Code Editor (index.html):

```
2 <html lang="en">
15 <body>
16 <header>
18 </header>
19 <section id="user-goals">
20 <h2>My course goals</h2>
21 <input type="text" v-model="currentGoal" />
22 <button @:click="addGoal">Add Goal</button>
23 <p v-if="goals.length === 0">No goals have been added yet - please start adding some!</p>
24 <ul v-else>
25 <li>Goal</li>
26 </ul>
27 </section>
28 </body>
29 </html>
```

Code Editor (app.js):

```
1 const app = Vue.createApp({
2   data() {
3     return {
4       goals: [],
5       currentGoal: "",
6     };
7   },
8   methods: {
9     addGoal() {
10      this.goals.push(this.currentGoal);
11      this.currentGoal = "";
12    },
13  },
14 });
15 app.mount("#user-goals");
```

Browser Preview:

- A green header box with the text "Vue Course Goals".
- A white box titled "My course goals" containing a text input field and a pink "Add Goal" button.
- A green rounded rectangle labeled "Goal".

Console:

vue.global.js:12572 You are running a development build of Vue. Make sure to use the production build (*.prod.js) when deploying for production.

Annotation: A red arrow points from the text "Vue JS uses v-if to show element conditionally, v-else element should be immediate neighbour of the v-if element" to the `<ul v-else>` tag in the HTML code.

Rendering Content Dynamically Using `v-show`

`v-show` show and hide element dynamically by adding css property to `display: none;`

```

<section id="user-goals">
  <h2>My course goals</h2>
  <input type="text" v-model="currentGoal" />
  <button @:click="addGoal">Add Goal</button>
  <p v-show="goals.length === 0">No goals have been added yet - please start adding some!</p>
  <ul>
    <li v-for="goal of goals">{{ goal }}</li>
  </ul>
</section>

```

Rendering List Items using v-for

v-for is used to render list

The screenshot displays a Vue.js application in a code editor and a browser. The code editor shows the following HTML and JavaScript code:

```

<html lang="en">
  <body>
    <header>
      </header>
    <section id="user-goals">
      <h2>My course goals</h2>
      <input type="text" v-model="currentGoal" />
      <button @:click="addGoal">Add Goal</button>
      <p v-if="goals.length === 0">No goals have been added yet - please start adding some!</p>
      <ul v-else>
        <li>Goal</li>
      </ul>
    </section>
  </body>
</html>

```

A red arrow points to the `v-else` attribute in the HTML code, with a note: "Vue JS uses v-if to show element conditionally, v-else element should be immediate neighbour of the v-if element".

```

const app = Vue.createApp({
  data() {
    return {
      goals: [],
      currentGoal: "",
    };
  },
  methods: {
    addGoal() {
      this.goals.push(this.currentGoal);
      this.currentGoal = "";
    },
  },
});
app.mount("#user-goals");

```

The browser shows the rendered UI, which includes a green header with the text "Vue Course Goals", a white box with the title "My course goals", a text input field, an "Add Goal" button, and a green box labeled "Goal".

The console shows a message: "You are running a development build of Vue. Make sure to use the production build (*.prod.js) when deploying for production."

Summary

Summary

Conditional Content

v-if (and **v-show**) allows you to render content **only if a certain condition is met**

v-if can be combined with **v-else** and **v-else-if** (only on direct sibling elements!)

v-for Variations

You can extract **values**, values and **indexes** or values, **keys** and indexes

If you need v-for and v-if, **DON'T use them on the same element**. Use a wrapper with v-if instead.

Lists

v-for can be used to render multiple elements dynamically

v-for can be used with arrays, objects and ranges (numbers).

Keys

Vue **re-uses DOM elements** to optimize performance → This can lead to bugs if elements contain state

Bind the **key** attribute to a unique value to help Vue identify elements that belong to list content